Leveraging Smaller Finite Fields for More Efficient ZK-Friendly Hash Functions

Gökçe Düzyol[1,2] and Kamil Otal[1]

[1] TÜBİTAK BİLGEM UEKAE
National Research Institute of Electronics and Cryptology
Gebze, Kocaeli, Türkiye
[2] Boğaziçi University, Bebek, Istanbul, Türkiye
{gokce.duzyol, kamil.otal}@tubitak.gov.tr

Abstract. Maximum distance separable (MDS) matrices are the main building blocks that provide the maximum possible diffusion in several block ciphers and cryptographic hash functions. In addition to using MDS matrices directly, there are also some indirect but simple and efficient methods that provide the maximum possible diffusion property. In particular, the subfield construction introduced by Barreto et al. in [DCC 56 (2-3) 141-162 (2010)] and its generalization examined by Otal in [IJISS 11 (2) 1-11 (2022)] make use of MDS matrices over smaller finite fields to provide the maximum possible diffusion property over larger finite fields.

ZK-friendly hash functions, in contrast to the classical cryptographic hash functions, use higher-dimensional MDS matrices over larger finite fields.

In this paper, we examine the applicability of the generalized subfield construction and the possibility of improvements on ZK-friendly hash functions. As a case study, we focus on a recent ZK-friendly hash function Vision Mark-32 presented by Ashur et al. in [IACR Preprint 2024/633]. In particular, instead of using a 24×24 MDS matrix over $\mathbb{F}_{2^{32}}$ for a 24×1 column input over $\{0,1\}^{32}$, we suggest separating the 24×1 column input over $\{0,1\}^{32}$ into four 24×1 subcolumns over $\{0,1\}^{8}$ and then using a 24×24 MDS matrix over $\mathbb{F}_{2^8}$ for each subcolumn. This method still keeps the maximum diffusion property without any compromise and provides simplicity and efficiency. For example, it is possible to significantly decrease the required LUT values to 265 from about 9200 and FF values to 102 from about 4600 for the hardware implementation. We also highlight that we do not need any additional tricks such as NTT for field multiplications.

We also push the theoretical boundaries of the generalized subfield construction to see how much small finite fields we can use, examine the arithmetization complexity, and discuss its applicability to other ZK-friendly hash functions.

Keywords: ZK-friendly hash functions · Arithmetization-oriented hash functions · MDS matrices · Generalized subfield construction.

1 Introduction

Zero-knowledge (ZK) proof systems are among the most popular and practical privacy-enhancing technologies. They have several applications in blockchains, cryptocurrencies, and web3 technologies. In particular, SNARKs (Succinct Non-Interactive Argument of Knowledge) have a special place due to their efficiency in verification and compact proof size.

A key challenge in ZK systems is the computational cost of hash functions. While traditional hash functions over binary fields, such as [6, 12], are optimised for computational efficiency, they impose high arithmetization costs, leading to inefficient performance in ZK applications. To address this, ZK-friendly (a.k.a. arithmetization-oriented) hash functions such as Poseidon [14], Rescue [2], Monolith [13], MiMC [1], Vision [2], Vision Mark-32 [4], Tip5 [34], XHash [3], and Polocolo [16] have been developed. In such designs, finite fields are generally chosen as either a large prime field or a large binary tower field of sizes around 2^{64}.

1.1 Motivation and Related Work

Sponge construction is preferred by many ZK-friendly hash functions and contains mainly the following round profile: The input is considered as an m-tuple of elements from a finite field $\mathbb{F}_q$ of size q. Then we apply the following steps in one round.

1. S-box layer: We apply the $x \mapsto x^d$ mapping for each entry x from the m-tuple, where d is a special small positive integer or -1. Then we evaluate x^d in a linearised or affine polynomial over $\mathbb{F}_q$.
2. MDS matrix multiplication layer: The m-tuple is then multiplied by an $m \times m$ MDS (maximum distance separable) matrix over $\mathbb{F}_q$.
3. Addition: We add a suitable m-tuple over $\mathbb{F}_q$.

The most costly part in this construction is the S-box layer, whereas the second one is the MDS matrix multiplication layer. For example, for Vision Mark-32 [4] which takes the parameters $q = 2^{32}$, $m = 24$, and $d = -1$ we can illustrate and compare the hardware implementation complexities in Table 1 below.

Table 1. [4, Table 3] Vision Mark-32 permutation round circuit complexity, implemented at 250 MHz.

Component	**LUT**	**FF**
S-box layer	41k	8.4k
MDS matrix multiplication	9.2k	4.6k
Total	50.0k	13.0k

Since the most costly part is the S-box layer, most of the performance-enhancing endeavours are focused on the S-box layer. However, as seen from

Table 1, nearly 20% of the required LUTs and 35% of FFs are consumed by the MDS matrix multiplication process; hence, the optimization for the MDS matrix multiplication layer has significant importance. In this paper, we focus on the efficient solutions for the MDS matrix multiplication process.

Historically, MDS matrices and, in general, MDS transformations have always been of interest since their usage in the Advanced Encryption Standard [11]. As a result of this interest, the last few decades have comprised intense research studies on the design and implementation of efficient MDS transformations, see for example [19, 33, 27, 20, 24].

However, cryptographically significant MDS matrices have been of dimensions mostly $m \times m$ where $4 \leq m \leq 8$ and over the finite fields of size mostly between $16 \leq q \leq 256$. Therefore, the research on MDS matrices has been dedicated to such lower dimensions and small fields.

On the other hand, for higher-dimensional MDS matrices over larger finite fields, there are only some generic methods such as the MDS matrices derived from Reed-Solomon (RS) codes (see, for example [30] for further details on RS codes). Other types, such as the circulant MDS matrices, are not very available for higher dimensions. Therefore, for higher dimensions and larger finite fields, it is not easy to systematically produce the most efficient MDS matrices. Instead, MDS matrices are randomly produced, and some implementation tricks such as Number Theoretic Transformation (NTT) are preferred for finite field multiplications (as in [4], for example).

Note that the motivation behind choosing MDS matrices comes from their maximum possible diffusion capability. However, the maximum possible diffusion property can be provided by various methods, direct usage of MDS matrices is not the only way. For example, there are some nonlinear [25] and additive [5, 27] MDS diffusion methods. We emphasize that especially the "generalized subfield construction" presented in [27] can be quite efficient for several ZK-friendly hash functions.

1.2 Our Contribution

In this paper, we examine the efficiency of the generalized subfield construction [5, 27] by studying Vision Mark-32 [4].

In particular, instead of using a 24×24 MDS matrix over $\mathbb{F}_{2^{32}}$ for a 24×1 column input over $\{0,1\}^{32}$, we suggest separating the 24×1 column input over $\{0,1\}^{32}$ into four 24×1 subcolumns over $\{0,1\}^{8}$ and then using a 24×24 MDS matrix over $\mathbb{F}_{2^8}$ for each subcolumn. This method still keeps the maximum diffusion property without any security and performance compromise (i.e., we do not need to increase the number of rounds or do anything else). Also, it is possible to significantly decrease the required LUT and FF numbers for the hardware implementation, for example, we implemented our idea with 265 LUTs and 102 FFs at clock frequency operating at 220 MHz (recall Table 1 which includes numbers about 9200 FFs and 4600 FFs for the hardware implementation). We also emphasize that we do not need any additional tricks such as NTT for finite field multiplications.

We also push the theoretical boundaries of the generalized subfield construction to see how much small finite fields we can use, examine the arithmetization complexity, and discuss its applicability to other ZK-friendly hash functions.

As a result, we show that the generalized subfield construction can be quite efficient; also, it provides simplicity and versatility.

1.3 Paper Organization

The rest of the paper is organized as follows: The next section gives preliminary information about MDS matrices and their efficiency, the generalized subfield construction, the round structure of the Vision Mark-32 ZK-friendly hash function, and basic information about FPGAs. Section 3 presents improvements based on the generalized subfield construction and regarding analysis. The last section contains some concluding remarks and future work.

2 Preliminaries

In this section, we give some basic information and fix our notation on MDS matrices and our target ZK-friendly hash function Vision Mark-32 [4].

2.1 MDS Transformations and Matrices

Let $\mathbb{F}_q$ denote the finite field of size q and $\mathbb{F}_q^l$ represent the set of l-tuples of $\mathbb{F}_q$. The function $d : \mathbb{F}_{2^n}^l \times \mathbb{F}_{2^n}^l \to \mathbb{R}$ given by $d(u, v) := |\{i : i \in \{1, 2, \ldots, l\}, u_i \neq v_i\}|$ satisfies the metric properties and is called the *Hamming distance*. A subset C of $\mathbb{F}_{2^n}^l$ endowed with the Hamming metric is called a *code*. We define the *minimum (Hamming) distance* of a code C by $d(C) := \min\{d(u, v) : u, v \in C, u \neq v\}$. There is an upper bound on $d(C)$ given by $d(C) \leq l + 1 - \log_{2^n}(|C|)$ and called the *Singleton bound*. A subset C of $\mathbb{F}_{2^n}^l$ satisfying the Singleton bound is called an *MDS code*.

The main parameters of a code C in $\mathbb{F}_{2^n}^l$ is denoted by $(l, |C|, d(C))_{2^n}$. In particular, if C is an m-dimensional subspace of $\mathbb{F}_{2^n}^l$ over $\mathbb{F}_{2^n}$, then we denote the main parameters of C by $[l, m, d(C)]_{2^n}$. MDS codes exist for many but not all parameters. In particular, the *MDS Conjecture*, which is stated below, formulates the existence of linear MDS codes for their parameters (see also [30, Conjecture 11.16] for example).

Conjecture 1 (The MDS Conjecture). The parameters of a linear MDS code $[l, m, l-m+1]_{2^n}$ of length l and dimension $m > 1$ over $\mathbb{F}_{2^n}$ satisfy the following:

$$l \leq \begin{cases} 2^n + 1 & \text{if } m \in \{2\} \cup \{4, 5, \ldots, 2^n - 2\}, \\ 2^n + 2 & \text{if } m \in \{3, 2^n - 1\}, \\ m + 1 & \text{if } m \geq 2^n. \end{cases}$$

A linear code corresponds to a rowspace of an $m \times l$ matrix over $\mathbb{F}_{2^n}$. Such a matrix is called a *generator matrix* of the code. In particular, linear MDS codes can be uniquely represented by a $m \times l$ matrix $[I : M]$, which is a concatenation of the $m \times m$ identity matrix I and $m \times (l - m)$ matrix M over $\mathbb{F}_{2^n}$. Here, the redundancy (check) part M of the generator matrix is called an *MDS matrix*. The minimum distance of a given linear MDS code equips the corresponding MDS matrix as in the following well-known result.

Proposition 1. *Let u be a nonzero $m \times 1$ matrix and M be a $m \times m$ MDS matrix over $\mathbb{F}_{2^n}$. Then the number of minimum total nonzero entries in both u and Mu is at least $m + 1$.*

Construction of MDS matrices is an important and intensely studied area in mathematics and cryptography. We refer the reader to [15], which is a recent and comprehensive survey in this area. We now give an important and generic construction method in the following proposition.

Proposition 2. *[26] For distinct $x_0, x_1, \ldots, x_{m-1}, y_0, y_1, \ldots, y_{m-1}$ from $\mathbb{F}_{2^n}$, the $m \times m$ matrix $A = (a_{i,j})$, where*

$$a_{i,j} = \frac{1}{x_i + y_j},$$

is an MDS matrix.

Remark 1 (Some Definitions and Facts). The matrix constructed in Proposition 2 is called *Cauchy matrix* of $x_0, x_1, \ldots, x_{m-1}, y_0, y_1, \ldots, y_{m-1}$. We remark that if $A = (a_{i,j})$ is a Cauchy matrix in this form, then for any two nonsingular diagonal matrices $D_1 = \mathrm{diag}(c_0, c_1, \ldots, c_{m-1})$ and $D_2 = \mathrm{diag}(d_0, d_1, \ldots, d_{m-1})$, the matrix

$$D_1 A D_2 = \left(\frac{c_i d_j}{x_i + y_j} \right)$$

is called a *generalized Cauchy matrix*. We emphasize the fact that if A is MDS, then $D_1 A D_2$ is MDS.

A generalization of the construction in Proposition 2 and its relation to Reed-Solomon codes are available below.

Proposition 3. *[32, 31] An $m \times 2m$ matrix G of the form $G = [I|A]$ over a finite field generates a generalized Reed-Solomon code if and only if $A = (a_{i,j})$ is a generalized Cauchy matrix, i.e.,*

$$a_{i,j} = \frac{c_i d_j}{x_i + y_j}$$

for $0 \leq i, j \leq m-1$, where the x_i, y_j are $2m$ distinct elements such that $x_i + y_j \neq 0$ for all i and j, and $c_i, d_j \neq 0$.

2.2 Efficiency of MDS Matrices

The efficiency of an MDS matrix is generally measured by the number of XORs that the mapping needs. In particular, we focus on the finite field multiplication between a constant finite field element and a variable. The number of XORs that the multiplication needs is determined by the irreducible polynomial which defines the finite field.

For example, consider the finite field $\mathbb{F}_{2^3} = \mathbb{F}(\alpha)$ where α is a root of $x^3 + x + 1 \in \mathbb{F}_2[x]$, the multiplication of $\alpha + \alpha^2$ and the arbitrary element $y = y_0 + y_1\alpha + y_2\alpha^2$ ($y_i \in \mathbb{F}_2$ for $0 \leq i \leq 2$) is given by

$$\begin{aligned}(\alpha + \alpha^2)y &= (\alpha + \alpha^2)(y_0 + y_1\alpha + y_2\alpha^2) \\ &= (y_1 + y_2) + (y_0 + y_1)\alpha + (y_0 + y_1 + y_2)\alpha^2,\end{aligned}$$

which corresponds to the mapping

$$(y_0, y_1, y_2) \mapsto (y_1 + y_2,\ y_0 + y_1,\ y_0 + y_1 + y_2). \tag{1}$$

Here, the addition corresponds to the XOR operation, and hence we need to apply $1 + 1 + 2 = 4$ XORs to execute the multiplication by $\alpha + \alpha^2$. The XOR counting procedure given above is also called the *direct XOR (d-XOR) counting* [18].

It is possible to define the number of XORs in a slightly different way. Let $f(x) = f_n x^n + f_{n-1}x^{n-1} + \cdots + f_1 x + f_0$ be an irreducible polynomial of degree n over $\mathbb{F}_2$, let α be a root of f, and define $\mathbb{F}_{2^n} = \mathbb{F}(\alpha)$. Here, considering α, define

$$M_\alpha = \begin{bmatrix} 0 & 0 & \dots & 0 & f_0 \\ 1 & 0 & \dots & 0 & f_1 \\ 0 & 1 & \dots & 0 & f_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & 1 & f_{r-1} \end{bmatrix}.$$

We call M_α the *companion matrix* of α over $\mathbb{F}_2$. There is a field isomorphism between $\mathbb{F}_{2^n}$ and $\{0\} \cup \{M_\alpha^i : 0 \leq i \leq 2^n - 2\} \subseteq \mathbb{F}_2^{n \times n}$. Here, the number of ones in M_{α^i} for some $\alpha^i \in \mathbb{F}_2(\alpha)$ and $0 \leq i \leq 2^n - 1$ is equal to n plus the d-XOR of α^i. For instance, taking $\mathbb{F}_{2^3} = \mathbb{F}(\alpha)$ where α is a root of $x^3 + x + 1 \in \mathbb{F}_2[x]$, we see that

$$M_\alpha = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

Also, see that the companion matrix of $\alpha^2 + \alpha$ is

$$M_{\alpha^2+\alpha} = M_\alpha^2 + M_\alpha = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}.$$

Here, the number of ones in $M_\alpha^2 + M_\alpha$ is 7 and $n = 3$, hence the number of d-XORs of $\alpha^2 + \alpha$ is $7 - 3 = 4$ (recall the d-XOR of equation (1)).

The d-XOR counting process can be directly extended to matrices in $\mathbb{F}_{2^n}^{m\times m}$: Let $A \in \mathbb{F}_{2^n}^{m\times m}$ and M_A denote the $nm \times nm$ companion matrix of A over $\mathbb{F}_2$ which is constructed by taking $(M_A)_{i,j} = M_{A_{i,j}}$ for all $1 \leq i, j \leq m$. Then the number of XORs of A is equal to the number of ones in M_A minus nm. An MDS matrix with fewer XORs is considered more efficient than the others. In general, an MDS mapping with fewer XORs provides similar efficiency benefits.

We refer the reader to [7–10, 17, 21, 28, 35] for various types of optimization techniques.

2.3 Subfield Construction

Now we define a sample bijection between vector spaces $\mathbb{F}_{2^8}$ and $\mathbb{F}_{2^3} \times \mathbb{F}_{2^5}$ over $\mathbb{F}_2$: Let $\mathbb{F}_{2^8} = \mathbb{F}_2(\theta)$ where θ is a root of an irreducible polynomial of degree 8, $\mathbb{F}_{2^3} = \mathbb{F}_2(\alpha)$ where α is a root of an irreducible polynomial of degree 3, and $\mathbb{F}_{2^5} = \mathbb{F}_2(\beta)$ where β is a root of an irreducible polynomial of degree 5 over $\mathbb{F}_2$. Then, for

$$u = u_{0,0} + u_{0,1}\theta + u_{0,2}\theta^2 + \cdots + u_{0,7}\theta^7$$

where $u_{0,i} \in \mathbb{F}_2$ for $0 \leq i \leq 7$, we define

$$\begin{aligned} u_1 &= u_{0,0} + u_{0,1}\alpha + u_{0,2}\alpha^2 \\ u_2 &= u_{0,3} + u_{0,4}\beta + u_{0,5}\beta^2 + u_{0,6}\beta^3 + u_{0,7}\beta^4 \end{aligned}$$

and hence construct a bijection

$$u \leftrightarrow (u_1, u_2).$$

(We sometimes use notation $u_1 \| u_2$ to denote (u_1, u_2).) In that way, we obtain

$$1 + \theta^2 + \theta^3 + \theta^4 + \theta^6 \quad \leftrightarrow \quad (1 + \alpha^2, 1 + \beta + \beta^3)$$

for example. This bijection idea can be easily extended from $\mathbb{F}_{2^8} \leftrightarrow \mathbb{F}_{2^3} \times \mathbb{F}_{2^5}$ to

$$\mathbb{F}_{2^{n_1+n_2+\cdots+n_s}} \leftrightarrow \mathbb{F}_{2^{n_1}} \times \mathbb{F}_{2^{n_2}} \times \cdots \times \mathbb{F}_{2^{n_s}}$$

for arbitrary positive integers n_i $(1 \leq i \leq s)$, and to matrices directly. (Remark that there always exists an irreducible polynomial of any degree over $\mathbb{F}_2$, see [22] for further details.)

We now give another method, first introduced in [5] and called the *subfield construction*, to satisfy the maximum byte-wise branching like in Proposition 1.

Proposition 4. *[5] Let n be an even integer, u be a nonzero $m \times 1$ matrix over $\mathbb{F}_{2^n}$, u_1 and u_2 be $m \times 1$ matrices over $\mathbb{F}_{2^{n/2}}$ such that $u = u_1 \| u_2$, and M be an $m \times m$ MDS matrix over $\mathbb{F}_{2^{m/2}}$. Then the minimum total nonzero entries in both u and $Mu_1 \| Mu_2$ is at least $m + 1$.*

In the subfield construction introduced in Proposition 4, we multiply the number of XORs by 2, since the matrix is applied on two separate semi-columns.

2.4 Generalized Subfield Construction

In this section, we generalize Proposition 4 and then investigate the efficiency of this generalization with respect to the XOR counting.

Proposition 5. *[27] Let u be a nonzero $m \times 1$ matrix over $\mathbb{F}_{2^{n_1+n_2+\cdots+n_s}}$, u_i be a $m \times 1$ matrix over $\mathbb{F}_{2^{n_i}}$ for $1 \leq i \leq s$, and $u = u_1 \| u_2 \| \dots \| u_s$. Let also M_i be a $m \times m$ MDS matrix over $\mathbb{F}_{2^{n_i}}$ for $1 \leq i \leq s$ and $v := M_1 u_1 \| M_2 u_2 \| \dots \| M_s u_s$ be the $m \times 1$ matrix obtained by concatenating $M_1 u_1, M_2 u_2, \dots, M_s u_s$. Then the number of total nonzero entries in u and v is at least $m + 1$.*

We would like to highlight the differences between Proposition 5 and Proposition 4:

- We do not have to use the same MDS matrix M to multiply with u_1 and u_2 as in Proposition 4; we can use different MDS matrices M_1 and M_2 to satisfy the MDS branching.
- Similarly, M as in Proposition 4 does not have to be over $\mathbb{F}_{2^{n/2}}$, it can be over just a smaller field. That is, n does not have to be even (or a multiple of a certain fixed number).
- Additionally, we do not have to split u into two as u_1 and u_2, we can split it into more pieces.

The following example illustrates Proposition 5.

Example 1. Let $m = 2$, $s = 3$, $n_1 = 2$, $n_2 = 3$, $n_3 = 3$, $\alpha \in \mathbb{F}_{2^{n_1}}$ be a root of $x^2 + x + 1 \in \mathbb{F}_2[x]$ and $\beta \in \mathbb{F}_{2^{n_2}} = \mathbb{F}_{2^{n_3}}$ be a root of $x^3 + x + 1 \in \mathbb{F}_2[x]$. Let also

$$M_1 = \begin{bmatrix} \alpha & 1+\alpha \\ 1 & 1 \end{bmatrix}, \quad M_2 = \begin{bmatrix} \beta+1 & \beta^2 \\ 1 & \beta \end{bmatrix}, \quad M_3 = \begin{bmatrix} 1 & \beta \\ 1 & 1 \end{bmatrix}.$$

Note that M_1, M_2 and M_3 are MDS matrices. Remark that we can separate each nonzero $u \in \mathbb{F}_{2^8}^2$ by $u = u_1 \| u_2 \| u_3$ for some $u_1 \in \mathbb{F}_{2^2}^2, u_2 \in \mathbb{F}_{2^3}^2$, and $u_3 \in \mathbb{F}_{2^3}^2$. The transformation

$$u \mapsto v = M_1 u_1 \| M_2 u_2 \| M_3 u_3$$

is an MDS diffusion, i.e. the number of nonzero bytes in both u and v is at least 3.

2.5 Vision Mark-32 and Its MDS Matrix

Vision [2] is a keyed permutation based on the Marvellous design strategy. Vision operates over $\mathbb{F}_{2^n}$ and each round consists of two steps that differ only in the linearized affine polynomial. Main input is an m-tuple from $\mathbb{F}_{2^n}$. We denote the input state of the i^{th} round by $S_i = (s_{i,0}, \dots, s_{i,m-1})$ where $s_{i,j} \in \mathbb{F}_{2^n}$. Each step in one round of Vision consists of three operations on the state:

- **Inverse function**: $\pi(s_{i,j}) = s_{i,j}^{-1}$.
- **Linearized affine polynomial**: $B(s_{i,j}) = \sum_{k=0}^{n-1} \beta_j s_{i,j}^{2^k} + \beta_n$.

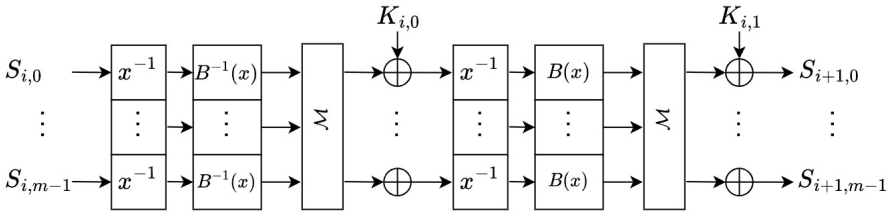
– **MDS matrix**: $L(S_i) = M \cdot S_i$.

The only difference between the two steps is the linearized affine polynomial. The linearized affine polynomial of the second step has the form:

$$B(x) = \beta_0 x + \beta_1 x^2 + \beta_2 x^4 + \beta_3,$$

which is a sparse polynomial. The linearized affine polynomial of the first step is B^{-1}, which is dense with a high degree. The round function of Vision is depicted in Figure 1.

Fig. 1. [4] One round of Vision with two steps



Vision Mark-32 is a hash function instantiating a Sponge construction using the Vision permutation. Vision Mark-32 has 8 rounds and operates over $\mathbb{F}_{2^{32}}$, with state size of $m = 24$.

In [23], a novel basis of polynomials over a finite field of characteristic 2 is introduced for efficient encoding and decoding of Reed-Solomon erasure codes [29]. The same basis is used in Vision Mark-32 to generate the MDS matrix.

2.6 Field Programmable Gate Arrays

A Field Programmable Gate Array (FPGA) is a reconfigurable integrated circuit that can be programmed post-manufacturing, unlike Application-Specific Integrated Circuits (ASICs), which are fixed at fabrication time. This flexibility makes FPGAs ideal for prototyping, design testing, iterative development and small-scale production, whereas ASICs are more suited for finalized, large-scale production.

FPGAs are categorized into families that share architectural characteristics but differ in performance and capacity. For instance, the Xilinx Artix-7 family offers a low-cost, resource-efficient platform suitable for embedded applications, while higher-end families like Virtex-7 cater to performance-intensive use cases. Each FPGA contains a variety of configurable resources—such as logic blocks, DSP slices, and memory—that enable implementation of custom hardware functionality.

Modern FPGAs consist of various configurable resources, each tailored to perform specific tasks efficiently. Understanding these resources is essential for designing optimized hardware architectures, particularly in cryptographic applications.

- **Flip-Flops (FFs):** FFs are fundamental storage elements used to hold single-bit values. While they offer the fastest data access times, their limited quantity makes them unsuitable for storing large datasets.
- **Look-Up Tables (LUTs):** LUTs are the primary logic elements in FPGAs, capable of realizing arbitrary Boolean functions by storing truth tables. Typically supporting 5 to 6 input bits, multiple LUTs can be combined for complex operations. They can also be repurposed as distributed memory (LUTRAM), though with slower access than FFs. Due to resource constraints, LUTRAMs are less efficient for storing large data structures like cryptographic keys.
- **Block RAMs (BRAMs):** BRAMs are specialized memory blocks integrated into FPGAs, designed for efficient storage of large data sets. In Xilinx devices, a typical BRAM has a capacity of 36 Kbits, which can either function as a single memory unit or be split into two 18 Kbit blocks. These blocks support various configurations for data width and depth, enabling flexible use—from storing thousands of single-bit values to handling wider data elements (e.g., 64 or 128 bits) by combining multiple BRAMs.
  BRAMs support several access modes that determine the number of simultaneous read and write operations. In single-port mode, one read and one write can occur sequentially in a clock cycle. Simple dual-port mode adds a second read port, while true dual-port mode allows two reads, two writes, or one read and one write per cycle. This versatility makes BRAMs highly suitable for performance-critical applications like post-quantum cryptography, where both speed and storage capacity are crucial.
- **DSP Blocks:** Digital Signal Processing (DSP) units enable high-speed arithmetic operations, particularly multiply-accumulate (MAC) computations. These are crucial for operations like matrix or polynomial multiplication and offer significant performance advantages over LUT-based implementations.

The availability and capacity of these resources vary significantly across FPGA families. While entry-level devices typically offer limited logic and memory, high-performance families provide substantially greater capabilities.

3 Improving the Performance of Vision Mark-32 by Leveraging the Generalized Subfield Construction

Vision Mark-32 [4] makes use of a 24×24 MDS matrix over $\mathbb{F}_{2^{32}}$. In general, it is not easy to find efficient MDS matrices in this form since:

- The dimension is too high. For example, there is no generic method to construct 24×24 circulant MDS matrices, probabilistic methods to produce

circulant MDS matrices can be cumbersome. Therefore, there are only some systematic constructions such as generalized Cauchy matrices mentioned in Proposition 3. However, we can not successfully control the number of XORs for all entries even if we can freely select the parameters c_i, d_i, x_i, and y_i in Proposition 3. Therefore, we suppose that each entry contains $\frac{n^2}{2} = 512$ ones and hence the matrix comprises $m^2 \cdot \frac{n^2}{2} - m \cdot n = 294144$ XORs on average.
– Also note that the finite field is too large. Therefore, the multiplication process requires some additional optimization ideas, such as NTT.

Instead of using a 24×24 MDS matrix over $\mathbb{F}_{2^{32}}$, we can apply the subfield construction given in Section 2.3. This time, the average number of XORs is

$$2 \cdot \left(24^2 \cdot \frac{16^2}{2} - 24 \cdot 16\right) = 146688.$$

Note that the expected number of XORs decreases to half.

3.1 Improvement 1

As observed and emphasized in [27], applying the generalized subfield construction by separating into more subcolumns may be more efficient. A direct idea is to use a 24×24 MDS matrix over $\mathbb{F}_{2^8}$ for each quarter column. In this way, the average number of XORs is

$$4 \cdot \left(24^2 \cdot \frac{8^2}{2} - 24 \cdot 8\right) = 72960.$$

See Figure 2 for an illustration of this method.

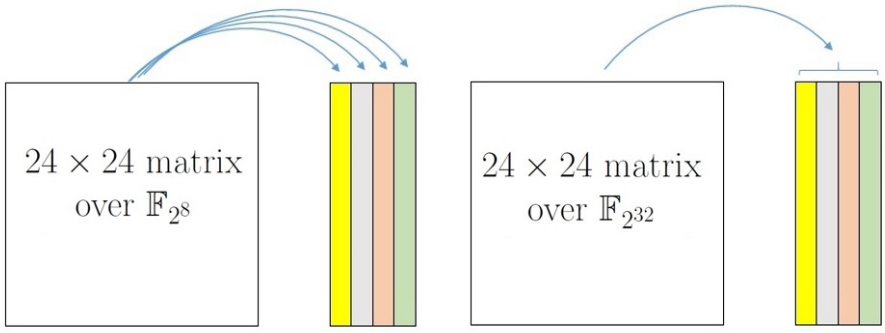
Note that this $8 + 8 + 8 + 8$ separation can be implemented by one matrix whose XOR complexity is 18240. In this way, the complexity of the implementation can decrease further. In reality, the complexity is much more smaller than we expected; we implemented a 24×24 MDS matrix over $\mathbb{F}_{2^8}$ by using Proposition 2 by only 265 LUTs and 102 FFs (recall the numbers in Table 1) at clock frequency of 220 MHz, achieving an execution time of 3.08 µs for a single matrix multiplication. In our implementation, no explicit optimization tricks such as NTT are required for the finite field multiplications, although the total execution time can be further reduced by increasing parallelism at the cost of slightly higher resource utilization.

It is, of course, possible to implement the matrix twice or four times and hence gain speed by compromising the area. In other words, the $8 + 8 + 8 + 8$ separation provides a performance gain in hardware implementation and a time-memory trade-off.

3.2 Improvement 2

The most aggressive separation is the $6 + 6 + 6 + 7 + 7$ version, since:

Fig. 2. Comparison of MDS diffusion methods



- There are no MDS matrices over binary finite fields of size smaller than 64 according to Conjecture 1.
- The Cauchy construction given in Proposition 2 is applicable for all binary fields of size $\mathbb{F}_{2^n}$, where $n \geq 6$.

By the $6+6+6+7+7$ separation, the average number of XORs is

$$3 \cdot \left(24^2 \cdot \frac{6^2}{2} - 24 \cdot 6\right) + 2 \cdot \left(24^2 \cdot \frac{7^2}{2} - 24 \cdot 7\right) = 58560.$$

It is possible to implement such a transformation by using about 1000 LUTs and 500 FFs at clock frequency operating at 220 MHz.

3.3 Additional Advantages and Comparisons

Another advantage of both improvements is that the time required for a multiplication in $\mathbb{F}_{2^8}$ (or $\mathbb{F}_{2^7}$ or $\mathbb{F}_{2^6}$) is much shorter than a multiplication in $\mathbb{F}_{2^{32}}$, hence we surely gain speed. So, we do not need special methods such as NTT for the multiplication, i.e. we also get the simplicity gain.

MDS matrices over $\mathbb{F}_{2^8}$ can be easy to apply since the MDS matrix of AES is over $\mathbb{F}_{2^8}$ too; i.e., there is a significant amount of available studies, codes, and optimizations on the operations over $\mathbb{F}_{2^8}$.

On the other hand, a possible advantage of 24×24 MDS matrices over $\mathbb{F}_{2^7}$ and over $\mathbb{F}_{2^6}$ is the following: It is possible to construct $\mathbb{F}_{2^7}$ and $\mathbb{F}_{2^6}$ using irreducible polynomials x^7+x+1 and x^6+x+1 over $\mathbb{F}_2$, respectively. Hence, the arithmetic operations may be less costly since both polynomials are more lightweight (the number of terms is the smallest and the gap is the maximum) when we compare with the most lightweight irreducible polynomial $x^8 + x^4 + x^3 + x + 1$ over $\mathbb{F}_2$ of degree 8. (Recall that we require modular reduction with an irreducible polynomial of degree n to run arithmetic operations in $\mathbb{F}_{2^n}$.)

It is of course possible to separate 32 as $6 + 6 + 6 + 6 + 8$ and hence use 24×24 MDS matrices over $\mathbb{F}_{2^8}$ and over $\mathbb{F}_{2^6}$ on 5 suitable subcolumns. The

possibilities can be increased by applying suitable separations such as $7+8+8+9$ or $10+10+12$, depending on the capabilities and requirements. This versatility shows a kind of algebraic richness and prospective trade-offs of the generalized subfield construction.

3.4 Security and Arithmetization Complexity Analyses

Note that the direct multiplication by an MDS matrix and the generalized subfield construction are both linear operations, and their unique purpose is to provide the maximum possible diffusion. They are both successful from this point of view, see Propositions 1 and 5. Also, it is interesting to examine whether any attacks can exploit generalized subfield construction and gain any advantage. We conjecture that no attacks obtain any advantage when we replace the direct MDS matrix multiplication with the generalized subfield construction.

The multiplication by an MDS matrix is a scalar operation. Therefore, using a different MDS matrix or the generalized subfield construction does not change the arithmetization complexity (see also [2, Section 4.1.5]).

3.5 Applicability to Other ZK-Friendly Hash Functions

The generalized subfield construction is applicable only when the finite field is not prime. Therefore, the ZK-friendly hash functions using large prime fields (such as Poseidon, MiMC, and Tip5) are disadvantageous than the ones using large binary finite fields (such as Vision, Starkad, and Vision Mark-32) from this perspective.

4 Final Remarks and Future Work

In conclusion, we observe that the generalized subfield construction provides the following advantages:

- **Efficiency:** There is a significant reduction on the number of XORs and FPGA implementation complexities.
- **Simplicity:** It is easy to implement and there is no need for additional optimization methods such as NTT.
- **Versatility:** Multiplication for each subcolumn can be implemented serially or in a parallel way, hence we can obtain a time-memory trade-off. Also, we can select the smaller fields freely (algebraic richness).
- **No security compromise:** We still keep the MDS diffusion feature. Also, there are no known attacks exploiting the properties of the (generalized) subfield construction.
- **No other efficiency compromises:** Arithmetization complexity does not increase.

Therefore, the generalized subfield construction is a powerful alternative, especially to higher-dimensional MDS matrices over larger finite fields.

It is also natural that several questions arise to understand and improve the generalized subfield construction method. We can list some of them below as future work:

- It is worth studying the applicability of the generalized subfield construction method (or similar ideas) when the base field is a prime field of a large prime size. In general, are there any (not necessarily linear but) efficient methods to satisfy the MDS diffusion property when the field is a (large) prime field?
- Investigation of different MDS matrix construction methods when the field is large and dimensions are high is another interesting question. For example, do 24×24 circulant MDS matrices exist over $\mathbb{F}_{2^8}$?
- Can generic methods (such as Cauchy MDS matrices) be improved for efficiency when the dimensions are high? For example, are there any probabilistic or deterministic methods to provide lighter (i.e., more efficient) 12×12 Cauchy MDS matrices over $\mathbb{F}_{2^6}$?
- We conjecture that no attacks obtain any advantage when we replace the direct MDS matrix multiplication with the generalized subfield construction. Note that it is not easy to prove such an assertion; however, can any attacks exploit the generalized subfield construction and gain any advantage?

References

1. Albrecht, M.R., Grassi, L., Rechberger, C., Roy, A., Tiessen, T.: MiMC: Efficient encryption and cryptographic hashing with minimal multiplicative complexity. In: Cheon, J.H., Takagi, T. (eds.) Advances in Cryptology - ASIACRYPT 2016 - 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4-8, 2016, Proceedings, Part I. Lecture Notes in Computer Science, vol. 10031, pp. 191–219 (2016). https://doi.org/10.1007/978-3-662-53887-6_7
2. Aly, A., Ashur, T., Ben-Sasson, E., Dhooghe, S., Szepieniec, A.: Design of symmetric-key primitives for advanced cryptographic protocols. IACR Trans. Symmetric Cryptol. **2020**(3), 1–45 (2020). https://doi.org/10.13154/TOSC.V2020.I3.1-45
3. Ashur, T., Kindi, A., Mahzoun, M.: XHash8 and XHash12: Efficient STARK-friendly hash functions. IACR Cryptol. ePrint Arch. p. 1045 (2023), https://eprint.iacr.org/2023/1045
4. Ashur, T., Mahzoun, M., Posen, J., Sijacic, D.: Vision Mark-32: ZK-friendly hash function over binary tower fields. IACR Cryptol. ePrint Arch. p. 633 (2024), https://eprint.iacr.org/2024/633
5. Barreto, P.S.L.M., Nikov, V., Nikova, S., Rijmen, V., Tischhauser, E.: Whirlwind: A new cryptographic hash function. Designs, Codes and Cryptography **56**(2–3), 141–162 (2010). https://doi.org/10.1007/S10623-010-9391-Y
6. Bertoni, G., Daemen, J., Peeters, M., Assche, G.V.: Keccak. In: Johansson, T., Nguyen, P.Q. (eds.) Advances in Cryptology - EUROCRYPT 2013, 32nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Athens, Greece, May 26-30, 2013. Proceedings. Lecture Notes in Computer

Science, vol. 7881, pp. 313–314. Springer (2013). https://doi.org/10.1007/978-3-642-38348-9_19
7. Boyar, J., Find, M.G., Peralta, R.: Small low-depth, low-size circuits for cryptographic applications. Cryptography and Communications **11**, 109–127 (2019). https://doi.org/10.1007/S12095-018-0296-3
8. Boyar, J., Matthews, P., Peralta, R.: On the shortest linear straight-line program for computing linear forms. In: International Symposium on Mathematical Foundations of Computer Science (MFCS) 2008. Lecture Notes in Computer Science, vol. 5162, pp. 168–179 (2008). https://doi.org/10.1007/978-3-540-85238-4_13
9. Boyar, J., Matthews, P., Peralta, R.: Logic minimization techniques with applications to cryptology. Journal of Cryptology **26**(2), 280–312 (2013). https://doi.org/10.1007/S00145-012-9124-7
10. Boyar, J., Peralta, R.: A new combinational logic minimization technique with applications to cryptology. In: International Symposium on Experimental Algorithms (SEA) 2010. Lecture Notes in Computer Science, vol. 6049, pp. 178–189 (2010). https://doi.org/10.1007/978-3-642-13193-6_16
11. Daemen, J., Rijmen, V.: The Design of Rijndael - The Advanced Encryption Standard (AES), Second Edition. Information Security and Cryptography, Springer (2020), https://doi.org/10.1007/978-3-662-60769-5
12. Gauravaram, P., Knudsen, L.R., Matusiewicz, K., Mendel, F., Rechberger, C., Schläffer, M., Thomsen, S.S.: Grøstl - A SHA-3 candidate. In: Handschuh, H., Lucks, S., Preneel, B., Rogaway, P. (eds.) Symmetric Cryptography, 11.01. - 16.01.2009. Dagstuhl Seminar Proceedings, vol. 09031. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, Germany (2009), http://drops.dagstuhl.de/opus/volltexte/2009/1955/
13. Grassi, L., Khovratovich, D., Lüftenegger, R., Rechberger, C., Schofnegger, M., Walch, R.: Monolith: Circuit-friendly hash functions with new nonlinear layers for fast and constant-time implementations. IACR Trans. Symmetric Cryptol. **2024**(3), 44–83 (2024). https://doi.org/10.46586/TOSC.V2024.I3.44-83
14. Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., Schofnegger, M.: Poseidon: A new hash function for zero-knowledge proof systems. In: Bailey, M.D., Greenstadt, R. (eds.) 30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021. pp. 519–535. USENIX Association (2021), https://www.usenix.org/conference/usenixsecurity21/presentation/grassi
15. Gupta, K.C., Pandey, S.K., Ray, I.G., Samanta, S.: Cryptographically significant MDS matrices over finite fields: A brief survey and some generalized results. Advances in Mathematics of Communications **13**(4), 779–843 (2019). https://doi.org/10.3934/AMC.2019045
16. Ha, J., Hwang, S., Lee, J., Park, S., Son, M.: Polocolo: A ZK-friendly hash function based on S-boxes using power residues (full version). IACR Cryptol. ePrint Arch. p. 926 (2025), https://eprint.iacr.org/2025/926
17. Jean, J., Peyrin, T., Sim, S.M., Tourteaux, J.: Optimizing implementations of lightweight building blocks. IACR Trans. Symmetric Cryptol. **2017**(4), 130–168 (2017). https://doi.org/10.13154/TOSC.V2017.I4.130-168
18. Khoo, K., Peyrin, T., Poschmann, A.Y., Yap, H.: FOAM: Searching for hardware-optimal SPN structures and components with a fair comparison. In: Batina, L., Robshaw, M. (eds.) Cryptographic Hardware and Embedded Systems - CHES 2014 - 16th International Workshop, Busan, South Korea, September 23-26, 2014. Proceedings. Lecture Notes in Computer Science, vol. 8731, pp. 433–450. Springer (2014). https://doi.org/10.1007/978-3-662-44709-3_24

19. Kranz, T., Leander, G., Stoffelen, K., Wiemer, F.: Shorter linear straight-line programs for MDS matrices. IACR Trans. Symmetric Cryptol. **2017**(4), 188–211 (2017). https://doi.org/10.13154/TOSC.V2017.I4.188-211
20. Kurt-Pehlivanoğlu, M., Sakallı, M.T., Akleylek, S., Duru, N., Rijmen, V.: Generalisation of Hadamard matrix to generate involutory MDS matrices for lightweight cryptography. IET Inf. Secur. **12**(4), 348–355 (2018). https://doi.org/10.1049/IET-IFS.2017.0156
21. Li, S., Sun, S., Li, C., Wei, Z., Hu, L.: Constructing low-latency involutory MDS matrices with lightweight circuits. IACR Trans. Symmetric Cryptol. **2019**(1), 84–117 (2019). https://doi.org/10.13154/TOSC.V2019.I1.84-117
22. Lidl, R., Niederreiter, H.: Introduction to Finite Fields and Their Applications. Cambridge University Press, Cambridge, UK (1994)
23. Lin, S.J., Chung, W.H., Han, Y.S.: Novel polynomial basis and its application to Reed-Solomon erasure codes. In: 2014 IEEE 55th Annual Symposium on Foundations of Computer Science. pp. 316–325 (2014). https://doi.org/10.1109/FOCS.2014.41
24. Liu, M., Sim, S.M.: Lightweight MDS generalized circulant matrices. In: Peyrin, T. (ed.) Fast Software Encryption - 23rd International Conference, FSE 2016, Bochum, Germany, March 20-23, 2016, Revised Selected Papers. Lecture Notes in Computer Science, vol. 9783, pp. 101–120. Springer (2016). https://doi.org/10.1007/978-3-662-52993-5_6
25. Liu, Y., Rijmen, V., Leander, G.: Nonlinear diffusion layers. Des. Codes Cryptogr. **86**(11), 2469–2484 (2018). https://doi.org/10.1007/S10623-018-0458-5
26. MacWilliams, F.J., Sloane, N.J.A.: The Theory of Error Correcting Codes. North-Holland Publishing Co., Amsterdam-New York-Oxford (1977)
27. Otal, K.: A generalization of the subfield construction. International Journal of Information Security Science **11**(2), 1–11 (2022), https://dergipark.org.tr/en/pub/ijiss/issue/70915/1104896
28. Paar, C.: Optimized arithmetic for Reed-Solomon encoders. In: IEEE International Symposium on Information Theory (ISIT) 1997. pp. 250–250. IEEE (1997)
29. Reed, I.S., Solomon, G.: Polynomial codes over certain finite fields. Journal of the Society for Industrial and Applied Mathematics **8**(2), 300–304 (1960). https://doi.org/10.1137/0108018
30. Roth, R.M.: Introduction to Coding Theory. Cambridge University Press (2006)
31. Roth, R.M., Lempel, A.: On MDS codes via Cauchy matrices. IEEE Trans. Inf. Theory **35**(6), 1314–1319 (1989). https://doi.org/10.1109/18.45291
32. Roth, R.M., Seroussi, G.: On generator matrices of MDS codes. IEEE Trans. Inf. Theory **31**(6), 826–830 (1985). https://doi.org/10.1109/TIT.1985.1057113
33. Sim, S.M., Khoo, K., Oggier, F.E., Peyrin, T.: Lightweight MDS involution matrices. In: Leander, G. (ed.) Fast Software Encryption - 22nd International Workshop, FSE 2015, Istanbul, Turkey, March 8-11, 2015, Revised Selected Papers. Lecture Notes in Computer Science, vol. 9054, pp. 471–493. Springer (2015). https://doi.org/10.1007/978-3-662-48116-5_23
34. Szepieniec, A., Lemmens, A., Sauer, J.F., Threadbare, B.: The Tip5 hash function for recursive STARKs (2023), https://eprint.iacr.org/2023/107
35. Visconti, A., Schiavo, C.V., Peralta, R.: Improved upper bounds for the expected circuit complexity of dense systems of linear equations over GF(2). Inf. Process. Lett. **137**, 1–5 (2018). https://doi.org/10.1016/J.IPL.2018.04.010